DSAC-EVaR: a practitioner's guide

Risk-seeking Entropic Value-at-Risk in an off-policy distributional actor–critic — what the algorithm is, how to set it up, and what is left to run

1 The algorithm in one page

Standard SAC, with two substitutions: the critic returns a *distribution* over returns instead of a scalar, and the actor maximises a *risk functional* of that distribution instead of its mean.

Risk functional (the only genuinely new part). For return samples Z and level α:

$$\mathrm{EVaR}_\alpha[Z] = \min_{x>0} G(x), \qquad G(x) = x\Big(\log \mathbb{E}\big[e^{Z/x}\big] - \log\alpha\Big), \qquad x = 1/\beta.$$

- G is strongly convex $\Rightarrow$ G' monotone $\Rightarrow$ **bisect G' in $\log x$.** 20 steps. Never Newton (see §4).
- Search interval **per row**: $[10^{-3}\,\mathrm{sd}(Z),\ 10^{3}\,\mathrm{sd}(Z)]$. Not a constant (see §4).
- **Detach x^*.** Envelope theorem gives exact gradients without unrolling the solver.
- If $p_{\max} \geq \alpha$, return $\max(Z)$ exactly — no interior optimum exists.

Algorithm 1 DSAC-EVaR update (one env step = one update)

Sample minibatch (s, a, r, c, s', d) from replay; $r_{\text{eff}} \leftarrow r - \lambda c$
$a' \sim \pi_\phi(\cdot|s')$ ▷ tanh-squashed Gaussian
$y_j \leftarrow r_{\text{eff}} + \gamma(1-d)\big(\min_{k\in\{1,2\}} Z^{\text{targ}}_{k,\tau_j}(s', a') - \eta \log \pi_\phi(a'|s')\big)$ ▷ d = **terminated** only
$\mathcal{L}_Z \leftarrow \sum_k \mathrm{QuantileHuber}\big(Z_k(s,a),\, y\big)$; update both critics
$\tilde{a} \sim \pi_\phi(\cdot|s)$; $z \leftarrow \big[Z_1(s,\tilde{a});\, Z_2(s,\tilde{a})\big]$ ▷ stack $(2B, K)$: one solve, not two
$\rho \leftarrow \rho_\alpha(z)$ ▷ EVaR via bisection, or mean/CVaR/Wang/...
$\mathcal{L}_\pi \leftarrow \mathbb{E}\big[\eta \log \pi_\phi(\tilde{a}|s) - \min_k \rho_k\big]$; update actor
$\mathcal{L}_\eta \leftarrow -\eta\big(\log \pi_\phi(\tilde{a}|s) + \bar{\mathcal{H}}\big)$; update temperature, $\bar{\mathcal{H}} = -\dim(\mathcal{A})$
Polyak-update targets, $\tau = 0.005$

The one structural requirement: the critic must be $Z(s,a)$, not $Z(s)$. EVaR is translation-equivariant, so $\mathrm{EVaR}(r + Z(s')) \equiv r + \mathrm{EVaR}(Z(s'))$ for a *sampled scalar* r — the tilt never touches the reward, and at a terminal step α drops out entirely. Measured exact regret: 86.35% with the state-value form, 0.00% with the action-value form. Everything else here is a detail; this one is the method.

2 Model setup

Component	Setting	Component	Setting
Critic	IQN, $Z(s,a)$, twin	Optimiser	Adam, 3×10^{-4} (all three)
hidden	$(256, 256)$ ReLU	Batch	256
embedding	128	Replay	10^6
$n_{\cos}$	64	γ	0.99
Huber κ	1.0	Polyak τ	0.005
K critic/target	32 / 32	Updates per env step	1
K risk	**64**	Warm-up (random actions)	2k–5k steps
Actor	tanh-Gaussian	Target entropy	$-\dim(\mathcal{A})$, auto-tuned
hidden	$(256, 256)$ ReLU	α (risk level)	0.1
$\log\sigma$ clamp	$[-20, 2]$	Bisection steps	20

Swap the risk functional, keep everything else. `--risk mean` is the risk-neutral control sharing critic, actor, data and seeds. Run it every time: it is the only thing that lets you attribute a difference to the risk measure rather than to the algorithm. Also available: `cvar`, `wang`, `cpw`, `entropic` (fixed β), `meanvar`.

3 Critical stats

Setting	Arm	Return	Paired gap	Reading
Lottery gridworld	state-value $Z(s)$	—	−86.35%	the formulation error
(exact, no learning)	action-value $Z(s,a)$	—	**0.00%**	exact at all 7 α
Pendulum-v1	`mean` s0/s1	−96.99 / −152.44	—	solved (≈ −1200 random)
(60k steps)	$\mathrm{EVaR}_{0.1}$ s0/s1	−96.36 / −151.92	+0.63 / +0.52	ties, correctly
SafetyPointGoal1	`mean`	26.99 (cost 56.3)	—	A2C never left random
(300k, $\lambda = 0$)	$\mathrm{EVaR}_{0.1}$	27.39 (cost 49.3)	+0.40	ties, environment's fault

Why the ties are not failures. Pendulum's return spread is policy noise, not risk in the world. On Safety-PointGoal1 the measured spread of the critic's distribution is $\mathrm{sd}(Z) \approx 0.022$ against episode returns near 27 — **under 1%**. With no spread there is no tail to trade, so every risk attitude ties by construction. Both operator health checks pass in the same runs (`at_bound_frac`$= 0$, `top_mass_frac`$= 0.017 \approx 1/K$), which is what makes that conclusion safe rather than a guess.

Throughput (A40, batch 256): Pendulum EVaR 75 steps/s vs `mean` 112; SafetyPointGoal1 49 vs 63. MuJoCo itself runs at 617 steps/s, so the learner is always the bottleneck — the EVaR solve costs 12% on CPU, the rest is GPU kernel-launch overhead.

4 Six things that will bite you

1. **$\alpha > 1/K$, always.** The top of K samples carries mass $1/K$; once that reaches α the dual runs to $x_{\min}$ and EVaR collapses onto $\max(Z)$. At $K = 64$ the floor is $\alpha > 0.0156$. Binds independently on the evaluation sample size.
2. **Never a fixed $[x_{\min}, x_{\max}]$.** x^* has *units of return* ($x^* \approx 0.34\,\mathrm{sd}(Z)$). A fixed interval works at one return scale and silently turns EVaR into mean $+\, x_{\min} \log(1/\alpha)$ — fixed-β entropic utility, i.e. your own baseline. Measured error up to 1100% at $\mathrm{sd}(Z) = 10^{-3}$.
3. **Bisection, not Newton.** When the tilted measure collapses onto one atom, $G'' = \mathrm{Var}_{Q_x}/x^3$ underflows, the step explodes, projects onto a bound, and cycles forever. Newton returned a *constant* $x^* = 0.301$ across different distributions and inflated EVaR by 25–590%.
4. **Mask zero-probability atoms to $-\infty$; do not `clamp_min(1e-12)`.** The tilt multiplies invented mass by $e^{z_{\max}/x}$ — a factor of e^{45} in a realistic case, worth +13.3% on EVaR.
5. **Size a categorical support in *discounted* units** if you use C51 at all. Undiscounted episode return left ≈ 9 of 51 atoms carrying mass. IQN avoids the problem entirely — prefer it.
6. **Leave `MUJOCO_GL` unset** for headless Safety-Gymnasium training. EGL or OSMesa in the same process as a CUDA context segfaults (rc 139). Lidar observations need no GL. Also: safety-gymnasium pins `gymnasium 0.28.1`, so it needs its own venv *with its own CUDA torch build.*

Instrument all of this. Log `at_bound_frac` (0 for $\alpha < 1$), `top_mass_frac` (near $1/K$), and `z_sd_mean` (says whether the environment can support the experiment *before* you spend a day on it). Each was added after the fault it detects had already cost real time. Validate the diagnostics themselves against known answers — our first bound-check compared the solution to itself and reported every solve as broken.

5 What remains: benchmarks and baselines

The evaluation gap is the whole gap. The operator is correct; what is missing is a setting where an *adaptive* tilt beats a fixed one, plus the standard comparisons. Framed as an agent × task × constraint matrix, in the style of GUARD (arXiv:2305.13681):

Suite	Instances	Why it matters here	Status
Safe Navigation (Safety-Gym)	PointGoal1/2, CarGoal, PointPush, PointButton, Circle	Stochastic hazards; needs $\lambda > 0$ to create return spread	PointGoal1 **done** rest **todo**
Safe Velocity	Hopper, HalfCheetah, Swimmer, Walker2d, Ant	What recent risk-sensitive papers report; direct comparability	**todo**
GUARD	8 agents (Swimmer, Ant, Walker, Humanoid, Hopper, Arm3, Arm6, Drone) $\times$ Goal/Push/Chase/Defense $\times$ 8 constraint types (Hazards, Ghosts, 3D variants)	Systematic difficulty sweep; 3D constraints and *interactive* (Chase, Defense) tasks are genuinely stochastic — the property our current envs lack	**todo**
Lottery gridworld	3-segment, exact EVaR	Ground truth; where C2 (vs SPSA) is actually runnable	**done**

Baselines still to implement (all against the shared `--risk mean` control): IQN with distortion measures (CVaR, Wang, CPW) — the standard risk-sensitive distributional comparison; fixed-β entropic utility — shows the dual solve earns its cost; CVaR-AC; DSAC proper; and WCSAC for the safety-constrained framing. **No SPSA head-to-head is planned**: the sample-efficiency claim is structural ($2N_t$ rollouts per update against one backprop) and the original paper's numbers stand for the SPSA side. The reference implementation is archived under `legacy/` if that is revisited.

Statistics. Current continuous-control numbers are 1–2 seeds and should be read as smoke tests. Publication needs ≈ 10 seeds with IQM and stratified bootstrap CIs (rliable), not mean $\pm$ CI over 5.

Next action. Sweep λ on SafetyPointGoal1 and re-screen with a 40k-step probe reading `z_sd_mean`. If pricing cost does not raise $\mathrm{sd}(Z)$ to a usable fraction of the return scale, no α sweep on that environment can mean anything, and the effort belongs in GUARD's interactive tasks instead.